Week 02 — Chapter I: Introduction to OS and System Programming

> Slides: week02-introduction-to-os-and-sp.pdf

Class overview

Grading	Attendance 20 (in-person), Assignments 30 (in-person), Midterm 30, Final 20
Platforms	Class materials on **LMS**; communication via **LMS and Kakao chatroom**

Supporting materials:

- Dhamdhere, D. M. (1993). *Systems Programming and Operating Systems*.
- Arpaci-Dusseau, R. H. & Arpaci-Dusseau, A. C. (2018). *Operating Systems: Three Easy Pieces* — free at https://pages.cs.wisc.edu/~remzi/OSTEP/
- Wienand, I. (2007). *Computer Science from the Bottom Up* — http://www.bottomupcs.com/

I.1. Overview

This chapter introduces generalities about **operating systems** and **system programming**, the important concepts, and their applications. It lays the foundation for the practical work in later chapters.

Outline

1. What is an Operating System (OS)?
2. Types of Operating Systems
3. Why Python for OS exploration?
4. What is System Programming (SP)?
5. Key areas in SP
6. System Programming in Python
7. Conclusion

I.2. What is an Operating System?

> An **Operating System (OS)** is a software program that **manages computer hardware and software resources** and **provides common services for computer programs**. It acts as an **intermediary between users and the computer hardware**.

System software is the category of programs designed to **operate, control and extend the capabilities of the hardware** (as opposed to application software, which solves a user's problem).

I.2.1. Main roles of an OS

Role	What it does
Process management	Manages the execution of multiple processes
Memory management	Allocates and deallocates memory for programs
File system management	Handles file storage, retrieval and permissions
Device management	Controls and coordinates hardware components
User interface	Provides a way for users to interact with the system (GUI or CLI)

The first two become whole chapters later: **process management** in week 03 and **memory management** in week 05.

I.2.2. Core components of an OS

Component	Description
Kernel	The **core part** of the OS; interacts **directly with the hardware**
Shell	The user interface that **takes commands and executes them** (Bash, PowerShell, ...)
File system	**Organizes and manages data storage** (NTFS, FAT32, ...)

I.3. Types of Operating System

Operating systems are classified by **design, functionality and usage**.

Type	Description	Examples
Single-user OS	Supports **one user at a time**, with limited multitasking	MS-DOS, Windows 95
Multi-user OS	Allows **multiple users** to access the system simultaneously	Linux, Unix, Windows Server
Batch OS	Executes jobs **in batches without user interaction**	IBM OS
Real-time OS	Processes data in **real time with strict timing constraints**	VxWorks, QNX
Distributed OS	Manages a group of **networked computers as a single system**	Amoeba, Plan 9
Embedded OS	Designed for **embedded systems with limited resources**	Embedded Linux
Mobile OS	Optimized for **smartphones and tablets**	Android, iOS

> ⚠️ In the slide, this table's **descriptions and examples are shifted out of alignment** (e.g. "IBM OS" appears beside the real-time row). The pairing above is the correct one.

Similarities and differences

Aspect	Similarities	Differences
Kernel	All OS have a kernel for hardware control	**Monolithic** (Linux) vs **microkernel** (QNX)
User interface	Most provide **GUI and CLI**	Some (e.g. servers) are **CLI-only**
Multitasking	Modern OS support multitasking	**Real-time OS prioritize task scheduling**
Security	All enforce user permissions	**Multi-user OS have stricter access control**

Monolithic vs microkernel: a monolithic kernel keeps most services (drivers, file systems) **inside the kernel** — fast, but a fault there affects everything. A microkernel keeps the kernel **minimal** and runs services as separate user-space processes — more robust and modular, with more communication overhead.

I.4. Why Python for OS exploration?

Python is widely used for **OS interaction** thanks to its simplicity and rich libraries, while **C** is traditionally used for **low-level OS development**.

Feature	Python	C
Ease of use	High-level, easy to learn and write	Low-level, complex memory management
Speed	Slower (interpreted)	Faster (compiled, direct hardware access)
Libraries	Rich ecosystem	Limited high-level abstractions
Portability	Cross-platform (works on Windows, Linux)	Requires recompilation for different OS
Use case	Scripting, automation, rapid prototyping	Kernel development, system programming

In this course Python is used to **observe and script the OS** (e.g. `psutil`, `os`, `platform`), not to build one.

I.5. What is System Programming?

> **System programming** is the development of software that **provides core services to the computer hardware** and **enables the execution of application programs**. It involves writing **low-level code** that interacts directly with the **OS, hardware and system resources**.

I.5.1. Main characteristics

- **Close to hardware:** deals with memory management, process control, device drivers.
- **Interacts with the OS:** uses **system calls** to request services from the kernel.
- **Performance-critical:** requires efficient management of CPU, memory and I/O.
- **Written in low/medium-level languages:** mainly **C, C++, Rust**, sometimes Python for scripting.

I.5.2. System programming vs application programming

Aspect	System programming	Application programming
Level of abstraction	Low-level (direct hardware/OS interaction)	High-level (user-facing software)
Languages used	C, C++, Assembly, Rust	Python, Java, JavaScript, C#
Purpose	Manages hardware and OS resources	Solves end-user problems (e.g. web apps)
Examples	OS kernels, device drivers, shells	Web browsers, games, office software

I.5.3. Key areas of system programming

Area	Examples
OS development	Writing kernels (Linux, Windows NT); implementing **system calls** (`fork()`, `exec()`, `open()`); **memory management** (paging, segmentation)
Device drivers	Software letting the OS communicate with hardware (printers, GPUs, USB devices); written in **C or Rust** for performance and hardware control
System utilities	Shells (Bash, PowerShell), debuggers (GDB), process monitors (htop)
Embedded systems programming	Firmware for microcontrollers (Arduino, Raspberry Pi); **RTOS** for robotics and automotive systems
Networking and security	Sockets, firewalls, VPNs; security modules (authentication, encryption)

System programming helps you:

- **Understand how computers work:** CPU, memory and I/O management.
- **Build efficient software:** optimize performance-critical applications.
- **Work on OS/embedded systems:** careers in kernel development, IoT, cybersecurity.
- **Debug complex issues:** diagnose low-level errors in applications.

I.6. Learning SP with Python vs C

Feature	Python (system scripting)	C (traditional system programming)
Level of abstraction	High-level (easy to use)	Low-level (direct hardware access)
Performance	Slower (interpreted)	Faster (compiled, optimized)
Use case	Automation, scripting, rapid prototyping	Kernel/driver development, real-time systems
Examples	`psutil` for process monitoring	Writing a Linux device driver

`psutil` is exactly what the week 03 lab uses for the CPU/memory dashboard.

I.7. Conclusion

OS and system programming are fundamental notions in computer science. This chapter covered the core information about both and their relevance, opening the way to deeper study in the following chapters.

Most important for exams

- **OS definition:** software that **manages hardware and software resources** and **provides common services** for programs; an **intermediary between users and hardware**.
- **Five roles:** process, memory, file system and device management, plus the **user interface**.
- **Three core components: kernel** (core, talks to hardware), **shell** (takes and executes commands), **file system** (organizes storage). Know an example for each.
- **Seven OS types** with description and example: single-user, multi-user, batch, real-time, distributed, embedded, mobile.
- **Monolithic (Linux) vs microkernel (QNX)**, and the GUI/CLI, multitasking and access-control differences.
- **System programming definition** and its four characteristics — especially that it **uses system calls** to request kernel services.
- **System vs application programming table** (abstraction, languages, purpose, examples).
- **Five key areas of SP:** OS development, device drivers, system utilities, embedded systems, networking and security. Know the example system calls `fork()`, `exec()`, `open()`.
- **Python vs C:** Python = high-level, interpreted, portable, scripting/automation; C = low-level, compiled, fast, kernels and drivers.